第四篇 · 执行引擎与动态编译

第 13 章 解释器、栈帧与异常执行

解释器把抽象操作数栈变成可执行的机器状态；异常则从当前方法的处理器表出发，逐层决定下一处继续执行的位置。

从现象开始

CH13-E01的leaf已经退出，main打印的异常栈仍有leaf。这个现象要求我们同时解释两件事：运行时怎样移除没有处理器的帧，以及异常对象怎样保留早先的调用信息。技术基线为Java开发工具包（Java Development Kit，JDK）25.0.5.0.2与macOS AArch64；本轮显式使用-Xint排除业务方法即时编译。

13.1 栈式模型与模板解释器

> 本节配图：CH13-F01。

13.1.1 规范状态怎样落到机器上

一个方法的字节码描述局部变量数组与操作数栈上的动作。以`iload_0; iconst_1; iadd`为例，语义是取局部值、压入常量、用两个整数替换为和。Java虚拟机规范（Java Virtual Machine Specification，JVMS）规定这些效果，却不要求每次动作都访问一块真实的内存栈。HotSpot模板解释器可以把栈顶值暂存在寄存器，用栈顶状态选择相应入口；只有需要时才把值写回表达式栈。栈式指令集与寄存器执行并不矛盾。

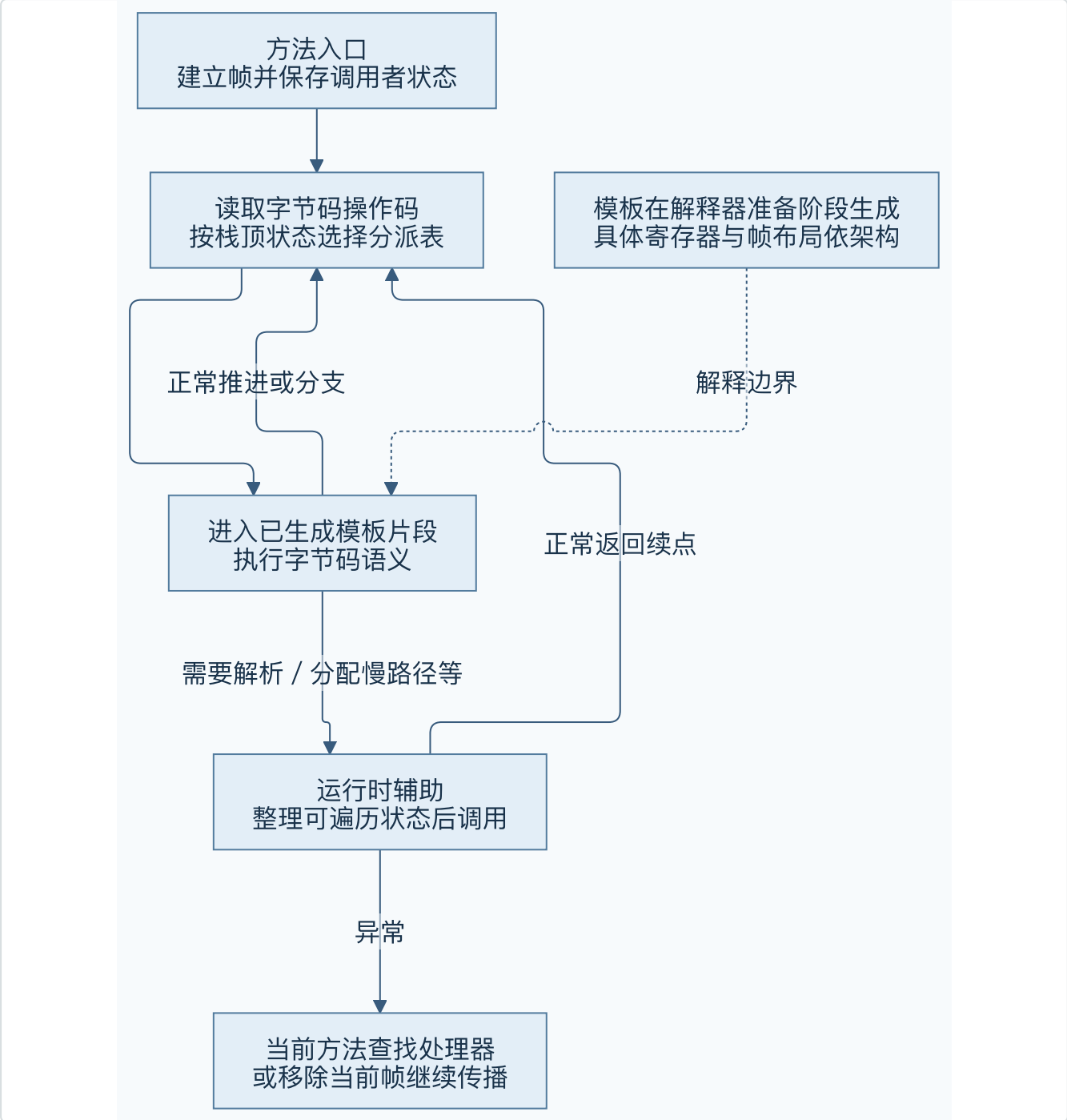
局部变量的第0槽在普通实例方法中承载接收者，静态方法没有这项隐含参数。long和double占两个规范局部槽；这条规则不能用来直接计算本地栈帧大小。第5章的栈映射用于验证类型状态，本章的运行时栈状态与验证用结构也不可混为一项。

13.1.2 模板不是逐条现编译

TemplateInterpreterGenerator在准备解释器时调用生成器，为不同字节码和栈顶状态生成本地代码片段，并登记入口。处理`iadd`时跳入现成片段执行加法，不重新启动Java编译器，也不逐次解释生成器的C++语句。模板生成器是生成这些机器片段的程序；Java字节码是这些片段要执行的输入。

与即时编译不同，模板通常围绕一个操作码的通用语义生成，不会据当前业务方法把整个循环和内联调用一起优化。实现仍可对字节码做内部重写、快速访问或特殊入口，因此“模板解释”也不等于对原始字节流完全

不做准备。以下图只保留主路径。



CH13-F01：解释器循环。机制示意，依据本章来源索引；箭头条件以图中文字为准。

13.2 入口、分派与模板生成

13.2.1 一次进入与多次分派

AArch64的generate_normal_entry先读取ConstMethod的参数数和局部变量数，检查栈空间、准备额外局

部值，再建立固定帧状态。方法、字节码位置和常量池相关状态让后续模板能找到当前语义。解释器宏汇编器dispatch_next推进字节码指针、读取操作码，并通过与栈顶状态对应的分派表间接跳转。分支指令则决定新的字节码位置，调用指令保存调用者状态后进入被调方法。

字节码位置索引（Bytecode Index，BCI）是相对方法Code字节数组的偏移。它不是本地程序计数器（Program Counter，PC）。一个字节码可以由许多机器指令承载，同一个模板又可服务许多不同方法；定位异常和调试状态时常需同时知道Method与BCI。

13.2.2 普通模板与辅助路径

简单算术可以在模板内完成；尚未解析的引用、分配慢路径或异常处理需要进入共享运行时。进入辅助调用前，机器状态必须整理成运行时能够遍历、识别引用和返回的位置；返回时再恢复解释状态。共享目录定义入口与策略，cpu目录提供寄存器、调用约定和机器指令实现，不能把某个AArch64寄存器名当作JVMS要求。

正常续点和异常续点不同。异常路径不会保留原表达式栈的任意中间值继续算术；它重新建立处理器入口状态，或者移除当前活动记录继续向调用者传播。这也是辅助调用不能被理解成一个普通C++函数返回后永远从原位继续的原因。

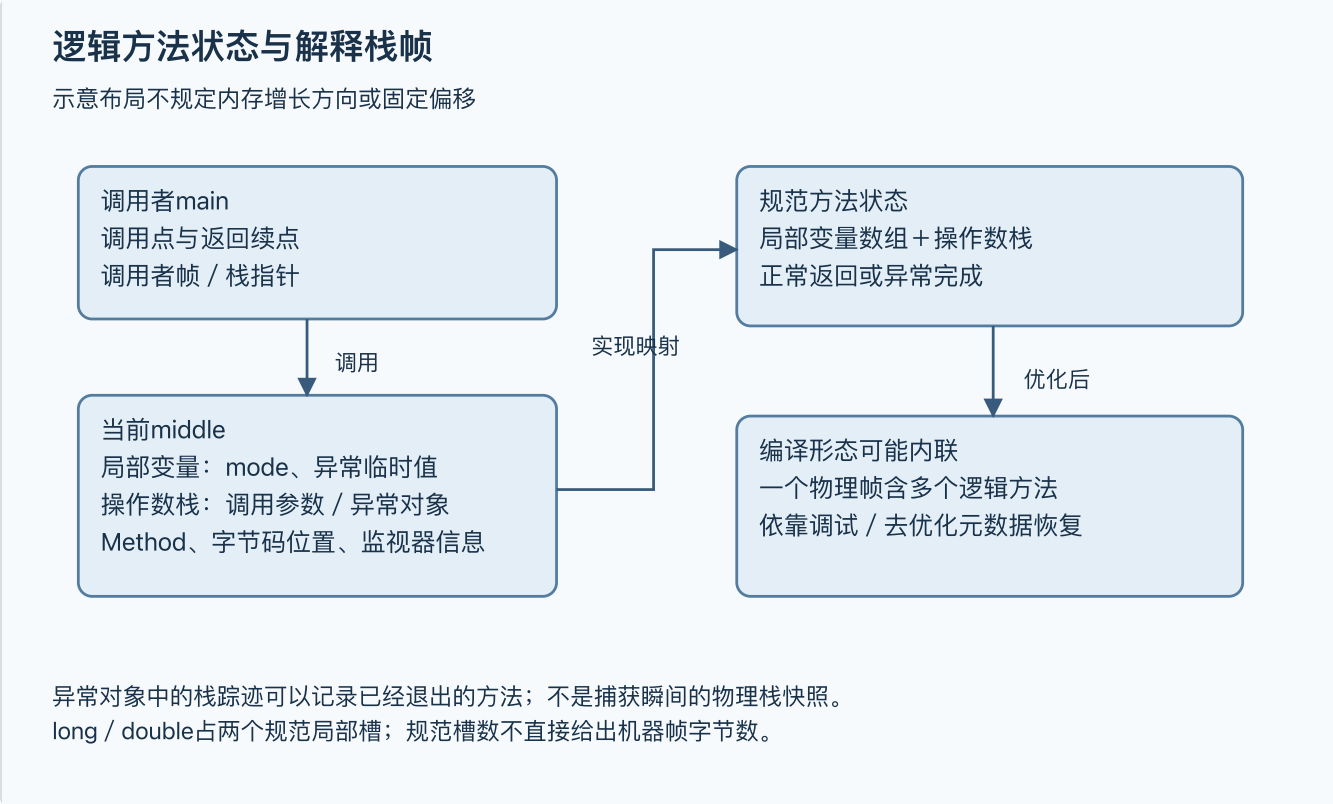
13.3 栈帧建立与遍历

> 本节配图：CH13-F02。

13.3.1 帧保存的是继续执行所需状态

解释帧保存参数与局部变量访问基址、表达式栈状态、当前方法和字节码位置，以及返回到调用者需要的信息。同步方法还需要相应监视器记录。被调方法正常完成时，结果按调用约定交回调用者，调用者从调用之后继续；异常完成则按调用点位置查找覆盖它的处理器。

遍历栈必须识别帧种类。解释器、编译代码、适配器和本地转换帧携带状态的方式不同；仅把栈内存按固定大小切块，无法还原调用链。第2章看到的系统线程栈与这里的Java方法活动不是两个独立不相关的栈集合；平台线程执行的本地帧承载着这些状态，但工具展示层次和覆盖范围各异。



CH13-F02：解释帧与逻辑栈。机制示意，依据本章来源索引；箭头条件以图中文字为准。

13.3.2 一层源码调用不保证一层物理帧

解释执行时，本例leaf、middle和main之间有清晰的调用关系。编译后leaf可能内联进middle，进一步并入main；诊断工具仍可利用元数据展示逻辑调用。这要求把“源码有三个方法”“工具显示三个栈项”和“机器上有三个独立帧”分开判断。

本例读取Throwable.getStackTrace发生在main捕获之后。此时leaf已退出；它仍出现在结果中，是因为异常对象通常在创建 / 填充栈踪迹时记录调用信息。重复抛出同一异常不会自动重新采集创建时栈。某些优化快速异常、隐藏帧过滤或显式关闭栈踪迹又会改变可见结果，因此栈踪迹不能用作任意运行模式下完整物理帧计数器。

13.4 异常传播

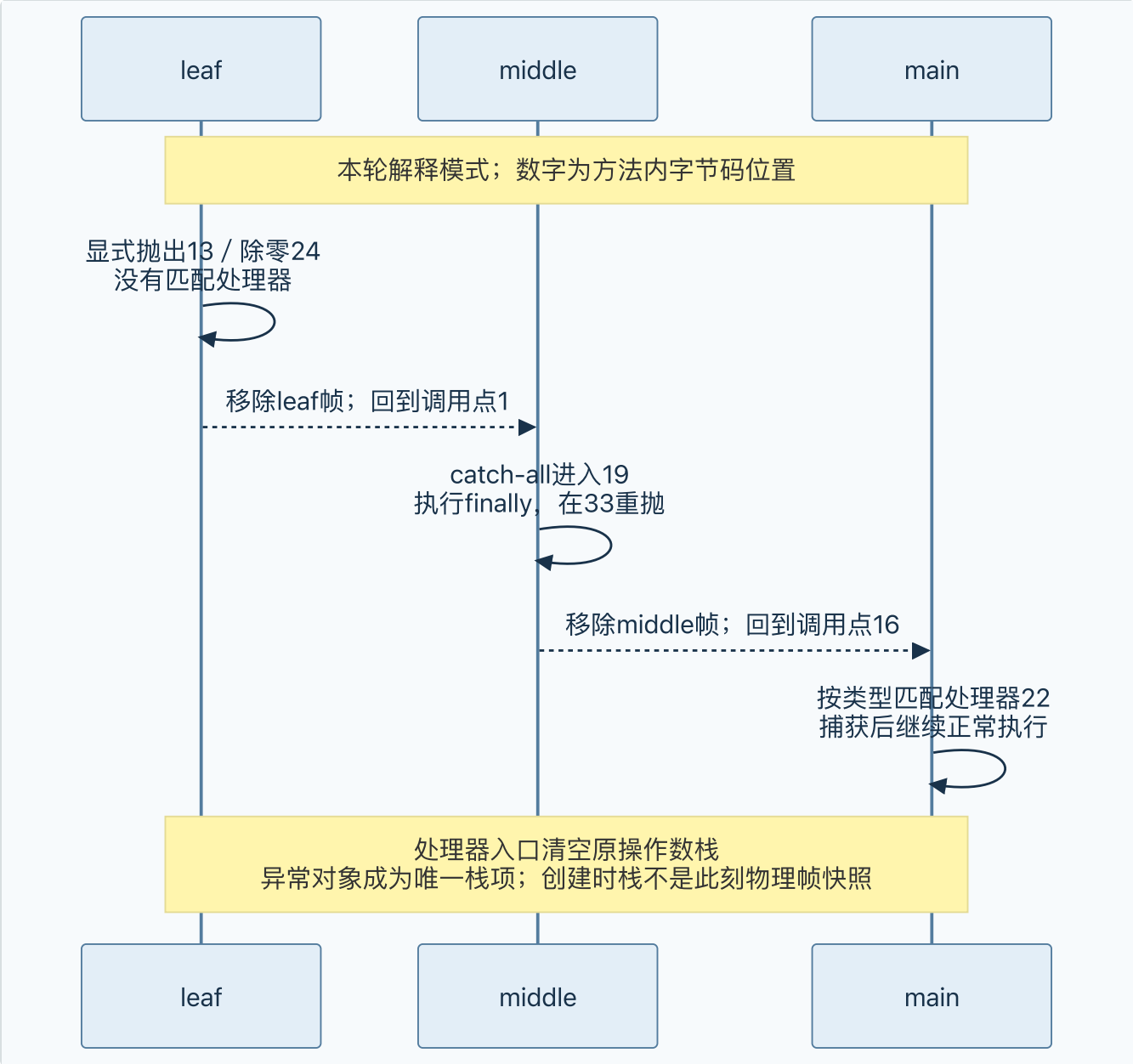
> 本节配图：CH13-F03。

13.4.1 在当前方法查找，再向调用者展开

本例显式创建IllegalArgumentException，另一分支执行整数除零。进入解释器异常入口后，HotSpot清空表达式栈，调用`InterpreterRuntime::exception_handler_for_exception`。它拿到当前Method、BCI与异

常类型，再通过`Method::fast_exception_handler_bci_for`查异常表。找到处理器后，把异常对象作为处理器栈上的唯一值，跳到处理器BCI。

若没有匹配项，返回移除活动记录的入口；释放需要解除的监视器、恢复调用者状态，再按调用点搜索。异常表范围是起点包含、终点不包含，类型为零的项匹配所有异常。main的多类型catch可以有多个异常表项指向同一处理器，而不是两个独立执行的catch。



CH13-F03：异常传播。机制示意，依据本章来源索引；箭头条件以图中文字为准。

本轮异常表与实际传播的对应

方法	覆盖 / 触发位置	处理器 / 下一步
leaf	显式抛出13，除零24	无处理器，展开到middle

方法	覆盖 / 触发位置	处理器 / 下一步
middle	调用点1落在保护范围	catch-all进入19，finally后33重抛
main	调用点16落在保护范围	两种异常项均指向22

13.4.2 显式抛出、运行时检查与最终退出

显式athrow要求把Throwable引用放到操作数栈；整数除零的idiv由执行实现建立ArithmeticException。两者进入共同的异常传播规则。本例middle的finally由javac生成catch-all处理器：保存异常、执行打印，再重新athrow。日志看到同一方法再次抛出，正是这一字节码结构；不能据此认定finally在虚拟机内部有一条专属指令。

若清理代码自身抛出新异常，新的传播会覆盖原有完成方式，除非源代码明确保留原因或抑制异常。若整个调用链均无处理器，则走线程未捕获异常处理与该线程退出；不必然终止全部进程。本例两次均在main内处理，退出码0，只观察语言机制，不模拟应用崩溃。

13.5 不同执行形态之间的转换

> 本节配图：CH13-F04。

13.5.1 解释器与编译代码靠适配器相接

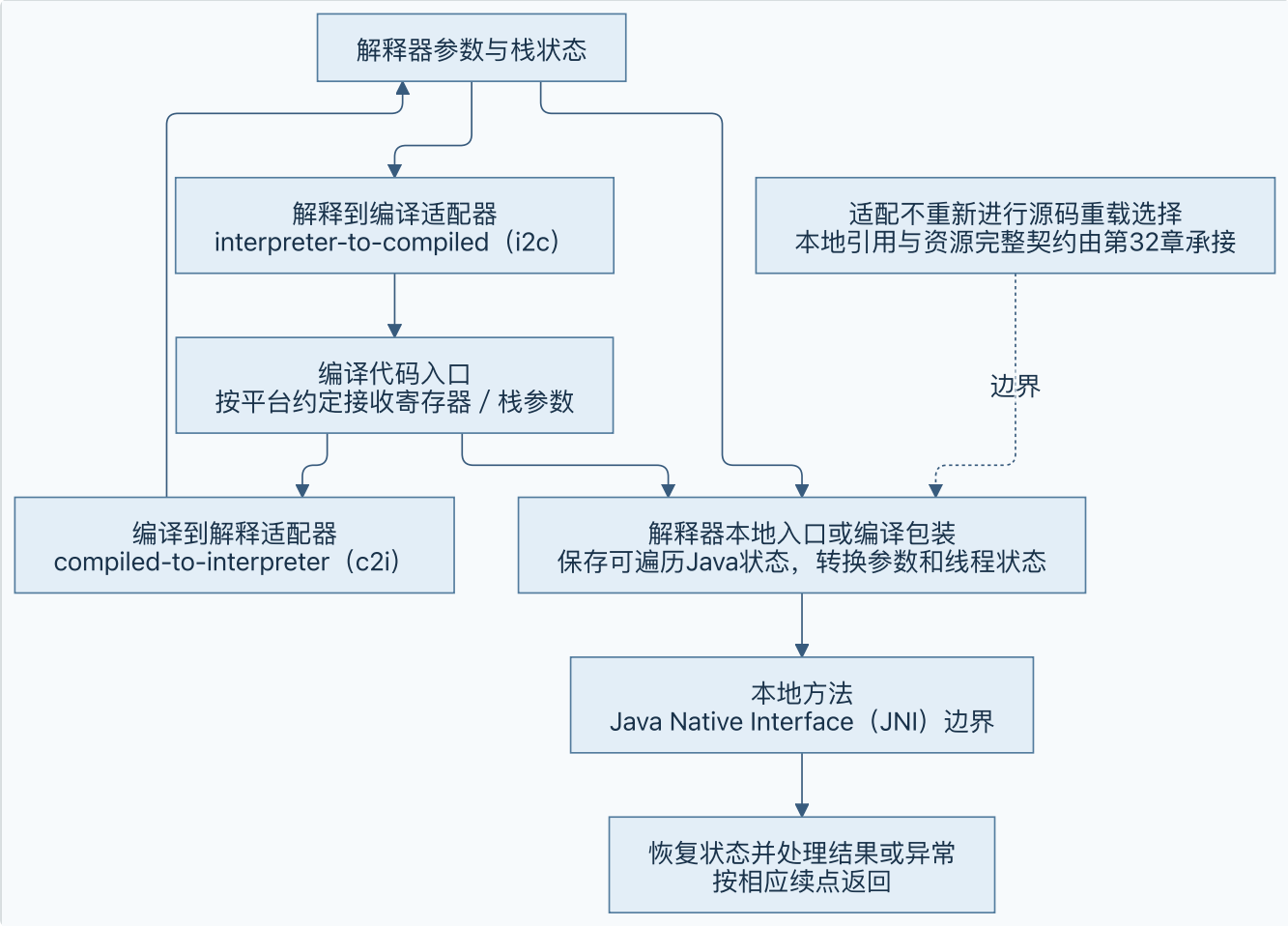
解释器习惯从表达式栈与局部变量取参数，编译代码按平台和类型分配寄存器 / 栈参数。SharedRuntime和AdapterHandlerLibrary按签名建立可共享适配器：解释到编译（interpreter-to-compiled，i2c）及编译到解释（compiled-to-interpreter，c2i）。适配器重新安排参数并接入目标入口，既不改变Java方法的声明类型，也不重新完成一次Java源码重载选择。

方法后来有了编译版本，运行时可以更新入口和调用点。某条正在执行的解释调用不会仅因为编译完成就凭空跳进机器码中间；普通入口切换与第18章栈上替换有不同的状态准备。调用者已编译也不要求所有被调方法同时编译。

13.5.2 本地边界还要保存线程状态

Java本地接口（Java Native Interface，JNI）调用需要本地包装与参数转换：Java对象引用不能作为任意裸地址长期保存，本地返回、异常和线程状态也需恢复。generate_native_entry与`SharedRuntime::create_native_wrapper`分别体现解释入口与编译包装侧的工作。进入本地代码前保存可识别的Java帧状态，运行时才知道如何从边界继续遍历。

这只是执行形态之间的接缝。句柄、线程附加、临界区和回调的完整所有权留给第32章；不能由本地方法看起来“只有一个调用”推导没有额外转换与协调成本。



CH13-F04：解释、编译与本地边界的适配。Mermaid图源与本地矢量导出见figures；图只说明执行形态接缝；完整本地资源契约仍归第32章。

证据与复做

实际运行两次依次打印中层finally、外层捕获、leaf→middle→main创建时栈，最后正常结束。解释器日志的leaf抛出位置分别为13与24，middle的catch-all位于19并在33重抛，main在22捕获；与javap.txt的异常表相符。所有位置都指本轮源文件，改变打印或表达式会改变BCI。

```
leaf: bci 13 / 24 → middle: bci 1 → handler 19
middle: bci 33 → main: bci 16 → handler 22
```

本轮编译与工具运行均退出0。记录目录为evidence/runs/ch13-20261103-01；未把本例栈踪迹推广为编译模式帧布局。

完整命令见运行说明，正式源文件的本地阅读副本见源码阅读。记录、输入身份及解释范围见证据索引。

回顾与边界

沿一条异常追踪时，先找抛出位置，再检查当前方法异常表；未匹配才向调用者展开。遍历结果若包含已内联或已退出的方法，需要核对工具重建和采集时点。下一章把调用指令展开为解析、选择和调用点策略，之后再回到编译与恢复。

来源

- S01：JVMS 25 §2.6、§2.10：帧与异常语义。
- S02：锁定OpenJDK提交1775ea1，templateInterpreterGenerator.cpp、AArch64模板生成器与interp_masm，精确符号见证据索引。
- S03：interpreterRuntime.cpp的exception_handler_for_exception与sharedRuntime.cpp适配器。
- O01：本章字节码、异常日志和标准输出；环境与限制见运行记录。